Digital Signal Processing Laboratory

Dylan Thornburg & Viet Ha

Laboratory 7: Real Time Implementations using Circular Buffers May 22, 23 2025

Prelab due Wednesday, 21 May 2025 at 8 pm

In the previous laboratory, low order FIR and IIR filters were implemented using small data storage buffers structured as software shift registers. These low order filters did not have enough frequency resolution to make frequency selective filters with well-defined band edges, and increasing the order for a higher performance filter requires a more efficient data structure.

In this laboratory you will implement data storage for input and output data using a circular buffer structure. This is a very efficient data structure used in a wide range of applications. A circular buffer uses a single contiguous block of memory to effectively store a continuous stream of data from which only a fixed maximum range of previous data is needed. This is accomplished by always addressing the data with an index modulo the length of the circular buffer, and because of the circular wraparound, no explicit tests for buffer overflow or underflow are needed. (If the buffer length is a power of two, the modulo arithmetic can be done with bit masking.)

Here we will use circular buffers for higher order bandpass and band stop filters with much better frequency resolution and narrower transition bands using Butterworth and Parks McClellen Filters designed in Laboratory 5. We will also use the structure to implement a very long filter with sparse coefficients and a time varying frequency modulation system.

Laboratory Procedure

Set up your board and connect it to the computer as you did in Lab 6.

Step 1: 4th order FIR test

- Download a new copy of the zip file and start a new project for lab 7.
- Copy the button response code from your lab 6 project to your lab 7 project.
- Add code to implement the general difference equation using the circular buffers from

your prelab.

- Create integer Nx, which will specify the number of the ***b*** coefficients used by your filter, which will be stored in xcoeff defined in the coefficients.h file.
  - For this first test, initialize Nx to 4.
  - Set the first four xcoeff values to 0.5, 0.0, 0.0, and 0.5.
- Create integer Ny, which will specify the number of ***a*** coefficients used by your filter, which will be stored in ycoeff defined in the coefficients.h file.
  - For this first test, initialize Ny to 1.
  - Set the first ycoeff value to 1.0.
  - Remember that if Ny is 1, you will have an FIR filter and you will NOT use any previous outputs in your calculation of a new output.
- This is filter 7 from your prelab 6 and the filter you ran in Step 6 of Lab 6.
- Write code in filter to get a new sample of data and save it in x(n), loop through the ***b*** coefficients and then the ***a*** coefficients (if Ny>1) to compute the new y output, then save the value of y in y(n), output it, and update n.

- When your program compiles without errors, test it and make sure it produces the same result that you observed in Step 6 of Lab 6 when you listen to the scales audio.

Checkpoint 1: Demonstrate your filter to your laboratory assistant.

Step 2: 2nd order IIR test

- Modify your code from Step 1 to set Nx = 1 and the first xcoeff value to 0.05. Set Ny = 2 and set the first two values of ycoeff to 1.0 and -0.95. (This is filter 4 from your prelab 6 and the filter you ran in Step 5 of Lab 6.).
- When your program compiles without errors, test it and make sure it produces the same result that you observed in Step 5 of Lab 6 when you listen to the scales audio. If your output sounds like it comes from a high pass filter rather than a low pass filter, what would cause this problem to happen?

Our output didn't sound like a high pass. It would've if we put +.95 instead of -.95.

- Save a listing of this program for your laboratory report.

```
#include "main.h"
#include "coefficients.h"
#include <stdbool.h>



#define XBUFFERLENGTH 512
```

```
#define YBUFFERLENGTH 32


extern const float xcoeff[XBUFFERLENGTH];
extern const float ycoeff[YBUFFERLENGTH];


float xnew;
float ynew;
float outVol;
bool outputEnb;
float xold, yold;
float xold2, xold3;




float getSample(void);
void send2OutBuffer(float);

//ADDED CODE FOR LAB 7

#define XBUF_LEN 512
#define YBUF_LEN 32
#define Nx 1 // # of b coeff
#define Ny 2 // # of a coeff

//General IIR Filter
float xbuffer[XBUF_LEN] = {0};
float ybuffer[YBUF_LEN] = {0};

int time_index = 0;

float filter_step(float input) {
	xbuffer[time_index] = input;
	float output = .0f;

	//current and past inputs (b coef)
	for (int i = 0; i < Nx; i++) {
		int index = (time_index - i + XBUF_LEN) % XBUF_LEN;
```

```
		output += xcoeff[i] * xbuffer[index];
	}

	//past outputs (a coef). also skip a0
	for (int i = 1; i < Ny; i++) {
		int index = (time_index - i + YBUF_LEN) % YBUF_LEN;
		output -= ycoeff[i] * ybuffer[index];
	}
	ybuffer[time_index % YBUF_LEN] = output;
	time_index = (time_index + 1) % XBUF_LEN;
	return output;
}

//END ADDED CODE



void leftButtonFunc() {
	outVol=outVol/1.26f;
}


void rightButtonFunc() {
	outVol=outVol*1.26f;
}

void upButtonFunc() {
	outputEnb=!outputEnb;
}

//void initFilter(void){
//	/*	Place your initialization code for this module here
//			This code runs once prior to any input, output, or filtering actions
//
//			It is a good place to initialize any data structures you create in this
module  */
//}

//void filter(void) {
//	/*	This is where you place all of your filtering code
//			This function is called 8K times per second prior to outputting any
sound samples to the DAC
//
```

```
//                          Ordinarily, each time this function is called you would:
//                                          read one data sample as input
//                                          perform any filtering or other computation to create
an output value
//                                          output the calculated value
//        */
//}

void initFilter(void) {
        xnew = .0f;
        ynew = .0f;
        xold = .0f;
        yold = .0f;
        xold2 = .0f;
        xold3 = .0f;
        outVol=1.0f;
        outputEnb=true;


}

void filter(void){
        xnew=getSample();
        ynew=filter_step(xnew);
        if(outputEnb){
                send2OutBuffer(outVol*ynew);
        }
        else{
                send2OutBuffer(xnew);
        }

}
```

Checkpoint 2: Demonstrate your filter to your laboratory assistant.

Step 3: Butterworth Bandpass Filter

- When the first two steps above are successful, a more complex filter is simply a matter of

using longer coefficient arrays for the computed coefficients and setting the values for Nx and Ny for the filter.

- Modify your program to implement the Butterworth Bandpass filter from the prelab. One way to transfer the coefficients computed in MATLAB to your C code is to use aString = strjoin(string(a), ',') for the ***a*** coefficient vector in MATLAB and paste it into your C code. Repeat for the ***b*** coefficients.
- Make sure that you set Nx and Ny to the correct values for this filter.
- Test the filter with the chirp audio input. You should clearly hear the frequency range of the passband, and the stopbands should be very quiet.

We hear the bandpass very clearly. The bandstops are attenuated nicely.

- Test it on the scales audio input. What notes of the scale do you hear in the passband?

We hear the middle notes, but it's hard for us to identify them as neither of us know music notes well. Should be C5 and C4.

- Test it on the song signal. How does it sound?

It sounds heavily filtered. Almost like it's underwater.

- Test it on the speech signal and note the effect.

Same effect. It's very hard to understand what Johnson is saying.

Checkpoint 3: Demonstrate your filter to your laboratory assistant.

Step 4: Butterworth Bandstop Filter

- Modify your program to implement the Butterworth bandstop filter by changing the coefficients in xcoeff and ycoeff.
- Test the bandstop filter with the chirp audio input. You should clearly hear the quiet range of the very narrow stopband.
- Test it on the scales audio input. What notes of the scale are missing because of the stopband?

The bandstop stops C6.

- Test it on the song and speech signals. What effect does the stopband have?

The stopband has little effect, because most frequencies aren't in its stopping range.

- Save a listing of this program for your laboratory report

Ycoeff:

1,-6.43222,20.5771,-42.0775,60.4562,-63.4748,49.2785,-27.9547,11.1415,-2.83846,0.359928

Xcoeff:

0.59994,-4.2951,15.2995,-34.7918,55.5072,-64.604,55.5072,-34.7918,15.2995,-4.2951,0.59994,

```
#include "main.h"
#include "coefficients.h"
#include <stdbool.h>



#define XBUFFERLENGTH 512



#define YBUFFERLENGTH 32


extern const float xcoeff[XBUFFERLENGTH];
extern const float ycoeff[YBUFFERLENGTH];


float xnew;
float ynew;
float outVol;
bool outputEnb;
float xold, yold;
float xold2, xold3;




float getSample(void);
void send2OutBuffer(float);

//ADDED CODE FOR LAB 7

#define XBUF_LEN 512
#define YBUF_LEN 32
#define Nx 11 // # of b coeff
#define Ny 11 // # of a coeff

//General IIR Filter
```

```
float xbuffer[XBUF_LEN] = {0};
float ybuffer[YBUF_LEN] = {0};

int time_index = 0;

float filter_step(float input) {
	xbuffer[time_index] = input;
	float output = .0f;

	//current and past inputs (b coef)
	for (int i = 0; i < Nx; i++) {
		int index = (time_index - i + XBUF_LEN) % XBUF_LEN;
		output += xcoeff[i] * xbuffer[index];
	}

	//past outputs (a coef). also skip a0
	for (int i = 1; i < Ny; i++) {
		int index = (time_index - i + YBUF_LEN) % YBUF_LEN;
		output -= ycoeff[i] * ybuffer[index];
	}
	ybuffer[time_index % YBUF_LEN] = output;
	time_index = (time_index + 1) % XBUF_LEN;
	return output;
}

//END ADDED CODE



void leftButtonFunc() {
	outVol=outVol/1.26f;
}


void rightButtonFunc() {
	outVol=outVol*1.26f;
}

void upButtonFunc() {
	outputEnb=!outputEnb;
}

//void initFilter(void){
```

```
//	/*	Place your initialization code for this module here
//		This code runs once prior to any input, output, or filtering actions
//
//		It is a good place to initialize any data structures you create in this
module  */
//}

//void filter(void) {
//	/*	This is where you place all of your filtering code
//		This function is called 8K times per second prior to outputting any
sound samples to the DAC
//
//		Ordinarily, each time this function is called you would:
//			read one data sample as input
//			perform any filtering or other computation to create
an output value
//			output the calculated value
//	*/
//}

void initFilter(void) {
	xnew = .0f;
	ynew = .0f;
	xold = .0f;
	yold = .0f;
	xold2 = .0f;
	xold3 = .0f;
	outVol=1.0f;
	outputEnb=true;


}

void filter(void){
	xnew=getSample();
	ynew=filter_step(xnew);
	if(outputEnb){
		send2OutBuffer(outVol*ynew);
	}
	else{
		send2OutBuffer(xnew);
	}
```

```
}
```

Checkpoint 4: Demonstrate your filter to your laboratory assistant.

Step 5: Parks McClellan Bandpass Filters

- Modify your program to implement the Parks-McClellen Bandpass filter of order 128.
- Test it on the chirp audio input. How does the sound compare to the Butterworth filter?

The sound is pretty comparable, but we can hear the stopband ripple.

- Test the Parks-McClellen Bandpass filter of order 64 on the chirp audio input. Can you hear the passband ripple of the filter? Is the stopband as quiet as the Butterworth filter output?This is noticeable on the chirp because it is sweeping through all the frequencies. It will be less noticeable on the scales audio.
  For the 64 order bandpass, we can hear the stopband ripples way more than on the order 128. The stopband also is just as quiet as the butterworth.

Step 6: Implement a filter to create an echo

Echoes occur when a signal can reach a single point by two paths that take different amounts of time. It can cause degraded audio quality in a room with hard walls that create strong reflections. Multipath in communications systems due to reflections from fixed structures and moving vehicles can increase error rates.

- Download a new copy of the zip file and start a new project for lab 7 echo.
- Copy the button response code from your lab 7 project to your lab 7 echo project.
- Increase the length of the xbuffer to 4096.
- Create two new variables – an integer delayLen and a float reflect.
  - Initialize delayLen to 4000 and reflect to 0.3.
- The reflection filter will be a very sparse filter. Almost all its coefficients will be zero, so implementation with the general difference equation method from the Lab 7 project used above would require unnecessary computation time.

- For each time index n, compute the output as
  ynew = (1-reflect)*x(n) + reflect*x(n-delayLen).
  Be sure to mask the indices for the xbuffer using the new buffer length.
- How would you expect the output of this filter to sound?
- Test it on the chirp and the scales input. You should hear two tones simultaneously due to the delay, but it will probably not sound like a distinct echo because when you are listening to pure tones, you don't have a sense of time localization.
- Test your echo program on the music and speech input and describe what you hear.

We hear just one echo.

- Change delayLen to 2000 and describe the effect on the music and speech input.

Harder to hear the echo now.

- Change delayLen to 800 and describe the effect on the music and speech input.

Now the echo just sounds more like reverb.

Checkpoint 5: Demonstrate your echo filter to your laboratory assistant.

Extra Credit Step 7 Modulation

If an input tone is multiplied point-by-point by a cosine function, the output will have two different tones. One will be the cosine frequency added to the input tone frequency and the other will be the cosine frequency subtracted from the input tone frequency. Clearly this is not an LTI system because the output has frequencies not present in the input.

- Add a sampled cosine buffer, cosMod, of length 2048 to your program and initialize the values computed with MATLAB to represent one cycle of a cosine
  – i.e. cosMod(n) = cos(2*pi*n/2048).
- cosMod will be used as a circular buffer to produce the modulating cosine value.
- Create a new float variable, modFreq, and initialize it to 200.0.
- Create a new integer variable, modStep.
  - In the initialization function, compute modStep to be the integer value of a step size for accessing the cosMod array that will produce a frequency closest to modFreq. i.e. You want cosMod(n*modStep) to produce sampled values of a cosine with frequency modFreq at an 8 kHz sample rate. You can do floating point computation of the value and then cast the result to an integer. (Hint: If modFreq were 4000, what should modStep be?)
- For time index n, compute the output as ynew = xnew*cosMod(modStep*n). Be sure to mask the index for cosMod using the array length of 2048.

- Test this on the scales input.
- Test it on the speech signal. How does the speech sound?

The speech sounds awful. The overlapped voices don't harmonize well.

- Change modFreq to 500.0 and test it on the speech signal. Then test with modFreq = 1000. Can you still understand the speech at these two modulation frequencies? How does it sound?

We could not really understand the speeches that well. The speaker sounds very warbly and alien almost.

Checkpoint 6: Demonstrate your modulator to your laboratory assistant.

Lab Report:

Submit the listings of your programs requested, a description of the observations requested, and answers to questions in the laboratory procedure.